

Copyright and AI Notice

These slides were presented at CppNow on 7th of May 2026 in Aspen, Colorado, USA. If you want to redistribute or adapt these slides, please check the license first.

No generative AI was used in the preparation of these slides.

©Hans de Nivelles, 2026

Implementing Logic in C^{++}

Hans de Nivelle

Nazarbayev University, Astana, Kazakhstan

Aspen, May 7th 2026

A Bit About Myself

- Have always worked in teaching and research.
- Currently working at Nazarbayev University, Astana, Kazakhstan.

Before, I have worked at University of Wrocław, Poland, and Max-Planck Institut für Informatik in Saarbrücken.

- Interested in logic and theorem proving. Have been using C^{++} , Java, and Prolog for implementation.

I like programming, probably a bit too much for academia.

About Nazarbayev University

- Nazarbayev University was established in 2010. Before that, the best students were sent to universities abroad.
- NU has 4000 undergraduate students, 1500 graduate students, and 700 students in the foundational year.
- Proficiency in English is a requirement for everyone.
- Kazakhstan is nice. The country is developing very quickly. Very big in surface area (2.7 million square km), but not so big in population (20 million). Currently, there are some problems, but they are not made in Kazakhstan.

It's All About Trees

Logical formulas are trees. One needs to construct, inspect and transform them.

I explain how to implement trees in C^{++} in such a way that they are 'easy' to use, and reasonably efficient.

Since the implementations are rather tedious, I use a source code generator (called **TreeGen**) to generate source code. This should be replaced by reflection in the future.

Using my approach, I have

- Implemented a complicated 3-valued logic called **PHOLI**.
- Implemented LISP (in compiler construction class).
- Implemented AST and the type system for a subset of C , in the same compiler construction class.
- Implemented compilers for research.

Some Existing Programs for Logic that Use C^{++}

Vampire: A theorem prover for first-order logic, written by Andrei Voronkov e.a. It is based on resolution.

Lean: A proof assistant used on dependent type theory, written by Leonardo de Moura, e.a. It has a built-in functional programming language. Programs in this language can be proven correct with help of the proof assistant.

Why Use C^{++} for Logic?

This is a small message to my colleagues who use C for theorem proving, but not C^{++} :

'But C^{++} is such a complicated language. I like the simplicity of C '

Yes, C^{++} is complicated, but you don't need to learn all of it.

There are few things that you can learn in a day, that will give you advantage immediately

Why C++?

- Function overloading: It is annoying that you have to call all your print functions `printX( )`, where `X` is the type being printed.
- Most data types that you are writing are of unpredictable size, and hence on the heap. In C++, heap management is **half-automatic** ('half-automatic' because the designer of a class needs to worry about it, but not the user of a class).

It is annoying that you have to define functions of form `copyX( )` and `cleanupX( )`, and remember when to call them. Look up 'RAII'.

- You can use namespaces.

You are silently giving up flexibility, and hence future functionality.

Example: First-Order Logic

The set of **first-order formulas** is recursively defined as follows:

- If p is a predicate symbol, $t_1, \dots, t_n$ ($n \geq 0$) are terms, then $p(t_1, \dots, t_n)$ is a formula.
- If t_1, t_2 are terms, then $t_1 \approx t_2$ is a formula.
- If F is a formula, then $\neg F$ is a formula.
- If $F_1, \dots, F_n$ are formulas, then $F_1 \wedge \dots \wedge F_n$, and $F_1 \vee \dots \vee F_n$, are formulas.
- If F_1 and F_2 are formulas, then $F_1 \rightarrow F_2$ and $F_1 \leftrightarrow F_2$ are also formulas.
- If $v_1, \dots, v_n$ is a sequence of variables, $T_1, \dots, T_n$ is a sequence of simple types of same length, and F is a formula, then $\forall v_1:T_1 \dots v_n:T_n \ F$ and $\exists v_1:T_1 \dots v_n:T_n \ F$ are also formulas.

Example: First-Order Logic (2)

- If v is a variable, then v is a term.
- If c is a constant, then c is a term.
- If $t_1, \dots, t_n$ are terms, and f is a function symbol, then $f(t_1, \dots, t_n)$ is a term.

There are also types, that I did not define.

Outline

- I show how to implement trees (nicely and efficiently). Trees can be logical formulas, proofs, mathematical expressions, or AST-s.
- I will introduce two variants of smart pointers.
- I will say something about allocation using `new`.
- I will explain what refinement types are, and why it would be nice to have them.
- If there is time left, I will say something about lists.

Code fragments can be found at

www.github.com/jeandenivelle/cppnow2026/

Symbolic Differentiation

I will use examples from symbolic differentiation.

Mathematical expressions are also trees, and the operations are similar (reconstruction and simplification).

In order to expose the difficulties with implementation, I need to use examples that are not simple. It is easier to do this with differentiation, because it is wider known.

On simple examples, every approach seems nice.

Differentiation Rules

- Basic Rules:

$$(\lambda x: x^c)' = \lambda x: cx^{c-1} \text{ if } c \text{ is a constant, and } c \neq 0.$$

$$(\lambda x: t)' = \lambda x: 0 \text{ if } x \text{ does not occur in } t.$$

- Arithmetic Rules:

$$(\lambda x: -f(x))' = -\lambda x: f'(x)$$

$$(\lambda x: f(x) + g(x))' = \lambda x: f'(x) + g'(x)$$

$$(\lambda x: f(x) - g(x))' = \lambda x: f'(x) - g'(x)$$

$$(\lambda x: f(x).g(x))' = \lambda x: f'(x).g(x) + f(x).g'(x)$$

$$(\lambda x: \frac{f(x)}{g(x)})' = \lambda x: \frac{f'(x).g(x) - f(x).g'(x)}{g(x).g(x)}$$

Differentiation Rules (2)

- Special Functions:

$$(\lambda x: \sin(f(x)))' = (\lambda x: \cos(f(x)).f'(x))$$

$$(\lambda x: \cos(f(x)))' = (\lambda x: -\sin(f(x)).f'(x))$$

$$(\lambda x: \exp(f(x)))' = (\lambda x: \exp(f(x)).f'(x))$$

$$(\lambda x: \ln(f(x)))' = (\lambda x: \frac{f'(x)}{f(x)})$$

I write $(\lambda x: f(x))'$ instead of $\frac{d(f(x))}{dx}$.

Differentiation is an operation on functions: It takes a function and returns a function.

Functions are formed with λ . It is the same λ that is written as `[]` in C^{++} . Every function definition is an implicit lambda.

Taking the derivative of $\lambda x: f(x)$ is the same as taking the derivative of $f(x)$ with respect to variable x .

What Choices do We have?

We want to implement symbolic differentiation in C^{++} .

We will do the implementation without the λ -s.

Define a function **derivative**(t_1, v) that returns a t_2 , s.t.

$$(\lambda v: t_1)' = \lambda v: t_2.$$

The choice space is big:

1. What is the format of the expressions?
2. Low-level representation. Under every implementation choice is a low level skeleton. We will look at these skeletons.
3. User Interface. How will the programmer interact with the tree data type?

Prolog

Let's have a look at another cool language: Prolog (Programmation en Logique)

- Everything is a tree. There are no types.
- Matching is built-in (actually unification).
- Control consists of backtracking and recursion.

We can have a look at **diff.pl**.

Mathematical Expressions

We define them as follows:

- Constant: (`rational`)
- Mathematical constants π and e .
- Variables: (`std::string`)
- Unary operator: $-$
- Binary operators: $-$ $/$
- N -ary operators: $+$ $*$
- Unary special functions: `sin`, `cos`, `exp`, `ln`, `sqrt`.
- Binary special function: `pow`

Adapting Some of the Rules

Since we made $+$ and $.$ (times) n -ary, we need to adopt the differentiation rules:

$(\lambda x: f_1(x) + \cdots + f_n(x))'$ equals $\lambda x: f'_1(x) + \cdots + f'_n(x)$.

$(\lambda x: f_1(x). \cdots . f_n(x))'$ equals

$$\sum \left\{ \begin{array}{l} f'_1(x).f_2(x). \cdots .f_n(x) \\ f_1(x).f'_2(x). \cdots .f_n(x) \\ \cdots \\ f_1(x).f_2(x). \cdots .f'_{n-1}(x).f_n(x) \\ f_1(x).f_2(x). \cdots .f_{n-1}(x).f'_n(x) \end{array} \right.$$

```

%define mathexpr ( sel_pi )
%option ratconst { sel_rat } => # val : rational
%option mathconst { sel_pi, sel_e } =>
%option var { sel_var } => # name : std::string
%option arith1 { sel_neg, sel_recipr }
    => sub : mathexpr
%option arith2 { sel_sub, sel_truediv }
    => sub1 : mathexpr, sub2 : mathexpr
%option arithN { sel_add, sel_mul }
    => [ sub : mathexpr ]
%option specfunc1 { sel_sin, sel_cos, sel_tan,
                    sel_exp, sel_ln, sel_sqrt }
    => sub : mathexpr
%option specfunc2 { sel_pow }
    => sub1 : mathexpr, sub2 : mathexpr

```

Every expression contains a tag (of type `selector`). The value of this tag defines the option, and the option defines the subfields.

- `mathexpr` is the type being defined. The tag between () defines the moved-out state. It should be an option without heap resources.
- Each option is followed by the selector values that have this option, between { }.
- After the `=>` are the fields of the option. Fields between [] are repeated. (Vector-like). Fields after a `#` are local (not on the heap).

We have decided the format of mathematical expressions. Note that we allow more than Prolog.

Next, we have to decide the following:

- Low level representation (skeleton).
- Sharing vs. non-sharing.
- User interface.

Low Level Representation

I assume only `selector` (an enumeration type), `std::string`, `rational`, and the type constructors `struct` and `union`.

Since the data structure is recursive, pointers are unavoidable. Pointers can be early or late.

I show in the code.

Something that does Not Occur in Differentiation

There is one thing that occurs in logic, but not in the differentiation example:

```
%define type ( type_form )  
%option prim { type_form, type_obj } =>  
%option struct { type_struct } => # def : exact  
%option unchecked { type_unchecked } => # id : identifier  
%option func { type_func } =>  
    result : type, [ arg : type ]
```

These are the structural types of PHOLI. In the last option, a type has form $T(T_1, \dots, T_n)$. The option contains a scalar field, and a vector field.

Since in C^{++} , derived classes must be below a pointer, I view derived classes as a variant of early pointer.

Early/late pointers are extremes. One can use compromises, or different layouts for different options. Non-recursive fields can be completely without pointer.

All this was about skeletons. It's not about what the user will see.

Pointers can be **shared** or **unique**.

Shared: Pro: Avoids deep copying, saves time and space. Con:
Puts value semantics at risk.

Unique: Pro: Value semantics guaranteed. Con: Deep copying
can be costly.

Some of the differentiation rules show that sharing is needed. This also applies to logic. **My Approach:** Use shared pointers with reference counting, but forbid non-const access to subtrees.

Wanted: C99-style flexible Arrays + Another new

I believe that one needs **structs** (on the heap) of the following form:

```
S scal;      // A single field of type S.  
R vect[];    // A repeated field of type R.
```

S could be `struct { size_t nrrefs; } or`
`struct { size_t nrrefs, size_t cap; size_t sz; }`

R can be anything (in our case `mathexpr`). It would be nice to have a new operator for it:

```
auto p = new { S(s), R[n] (r) };  
    // Gives you one S, initialized with s,  
    // and n R-s, initialized with r.  
delete[] p;  // Should work as usual.
```

Dubious Solution

No instances of this class are constructed:

```
template< size_t s > struct onheap
{
    size_t nrrefs;    size_t sz;    size_t cap;
    S scal; V vect[s];
};
```

Gnu++ rejects `onheap<0>`. Not sure why. Because of this, I also have:

```
struct onheap0
{
    size_t nrrefs;    size_t sz;    size_t cap;
    S scal;
};
```

Dubious Solution (2)

```
onheap0* ptr;

using SV = onheap<1> ;
constexpr static size_t offset0 = offsetof( SV, vect );
    // This is a non-portable C-makro. Would be nice
    // if a portable solution existed.

static constexpr V* vectbegin( onheap0* ptr )
    { return (( onheap<1> * ) ptr ) -> vect; }

static constexpr V* vectend( onheap0* ptr )
    { return vectbegin( ptr ) + ( ptr -> sz ); }
```

I will say more about implementation, but first the interface:

User Interface

We need to consider (1) construction, (2) inspection, and (3) updates.

Construction is easy. (Thanks to overload resolution and concepts!)

```

mathexpr( selector sel, const rational& );
mathexpr( selector sel, const std::string& );
mathexpr( selector sel );
mathexpr( selector sel, const mathexpr& );
mathexpr( selector sel,
          const mathexpr& , const mathexpr& );

template< std::random_access_iterator It > requires
std::convertible_to< std::iter_value_t<It>, mathexpr >
mathexpr( selector sel, It begin, It end );

mathexpr( selector sel,
          std::initializer_list< mathexpr > repeated );

```

Selectors can be checked dynamically, but unfortunately not statically! C^{++} has no refinement types.

Semi Constructors

For convenience, one can add semi-constructors:

```
inline mathexpr mathexpr_rat( const rational& val )  
    { return mathexpr( sel_rat, val ); }
```

```
inline mathexpr mathexpr_pi( )  
    { return mathexpr( sel_pi ); }
```

```
inline mathexpr mathexpr_neg( const mathexpr& sub )  
    { return mathexpr( sel_neg, sub ); }
```

```
inline mathexpr mathexpr_sub( const mathexpr& sub1,  
                               const mathexpr& sub2 )  
    { return mathexpr( sel_sub, sub1, sub2 ); }
```

Semi Constructors (2)

```
template< std::random_access_iterator It >
requires
std::convertible_to< std::iter_value_t<It>, mathexpr >
inline mathexpr mathexpr_add( It begin, It end )
    { return mathexpr( sel_add, begin, end ); }

inline mathexpr
mathexpr_add( std::initializer_list< mathexpr > args )
    { return mathexpr_add( args.begin(), args.end()); }
```


Inspection

Suppose we are given a `mathexpr`. We have to determine its option, and access the fields. Which fields exist depends on the option.

The following approaches exist:

visitor: Useful in functional programming, when you want to treat all options at once. Frequently, you want to deal with only one option (or one selector value), and fall through in other cases. It is also inconvenient to use nested cases.

It is too rigid. Sometimes you want to treat different selector values together, sometimes you want to treat them separately.

You can write your own visitors if you want.

matching: Nice, but C^{++} does not support it. Also, not that flexible. I wouldn't know how to extend it to vector fields.

You can write it if you want, but it will be not native.

Variables have to be managed in maps.

Inspection (2)

OO-style overriding: Expose the options (as subclasses) to the user and allow overriding.

Algorithms get scattered through different subtypes. This is OK for some algorithms, but most of the time it's not. It would be problematic with nested rules.

Classification always has to follow the subtype hierarchy.

Classes generated by TreeGen can be inspected in two ways:

1. Look at the selector (`switch` or `if`).
2. Directly check the option.

In both cases, one needs to create a view to the option. This is **seriously ugly**, but I think it can be avoided only with **refinement types**, which C^{++} does not have.

Inspection (3)

Using access method (1) (in the differentiation function):

```
switch( expr. sel( ))
{
case sel_add:
    {
        auto arN = expr. view_arithN( );
        auto res = mathexpr_add( { } );
        for( size_t i = 0; i != arN. size( ); ++ i )
        {
            res. view_arithN( ). push_back(
                deriv( arN. sub(i), var ));
        }
        return res;
    }
}
```

Inspection (4)

Using access method **(2)** (when printing a `mathexpr`):

```
if( expr. option_is_arith2( ))
{
    auto ar2 = expr. view_arith2( );
    out << expr. sel( ) << "( ";
    out << ar2. sub1( ) << ", " << ar2. sub2( );
    out << " )";
    return out;
}
```

The cast could be included in `option_is_arith2( )`. In that case, one could write a method

`if( auto ar2 = expr. try_arith2( ))`. Since I nearly always use access method 1, I didn't bother to write it. I feel sorry now.

Updating Subterms

Assigning to fields below a pointer is problematic, due to possible sharing.

This is the C^{++} approach to field assignment:

1. Give out the field as non-const reference.
2. The assignment operator (of the type of the field) assigns a value to the reference.

The Python approach works without references:

Updating a field is a method of the class being updated (called `__setattr__` for fields, and `__setitem__` for indexed things.)

We use the Python approach:

`expr. update_X( val )` for scalar field X.

`expr. update_X( i, val )` for vector field X.

Copy-On-Write (COW)

When an update-method is called, and the updated field is shared, make a unique copy.

More precisely: Whenever a part of a tree is updated, make sure that all pointers on the path to this tree are non-sharing.

This guarantees value-semantics.

Problem: In logic (and also in the differentiation example), many updates don't change the value.

Simplification:

```
simplify( 0 * _X, 0 ).  
simplify( _X * 0, 0 ).  
simplify( - - X, X ).  
simplify( - 0 , 0 ).
```

Most simplification attempts will have no effect.

Copy-on-Change

Recursively go through the term, and do (for example);

```
if( option_is_arith2( ))
{
    auto ar2 = expr. view_arith2( );
    ar2. update_sub1( simplify( ar2. sub1( )) );
    ar2. update_sub2( simplify( ar2. sub2( )) );
}
```

In most cases, simplification will have no effect, but the expression will still be reallocated.

Solution: Reallocate only when the new subtree differs from the old subtree

How to detect? Deep comparison is costly.

Strong Equality

Java has the `==` operator. Primitive types are compared by value, objects are compared by reference. This is possible, because in Java, every object is below a pointer. For value equals, Java uses the `.equals( )` method.

Python uses `is` for object identity, and operator `==` for value equality.

We can't use this here, because not everything is below a pointer.

Strong Equality: Compromise between `==` and `.equals( )`.

Compare values until the first pointer encountered, then compare the pointers.

Update

The update method is implemented as follows:

```
update_X( expr newval ) // Assuming X is below pntr
    if strongly_equal( pntr -> X, newval )
        return;

    if pntr -> nruses ) != 1
        -- ( pntr -> nruses )
        pntr = copy( pntr )
        pntr -> nruses = 1
    pntr -> X = newval
```

One could add update operations that always update (without checking for strong equality). It's not worth it.

Updates that return a term that is equal, but not strongly equal, are unlikely.

Rewriting (Simplification)

We want to walk through an expression, and make simplifications wherever possible. In Prolog:

```
simplify( X + 0, X ).
```

```
simplify( 0 + X, X ).
```

```
simplify( 1 * X, X ).
```

```
simplify( X * 1, X ).
```

```
simplify( 0 * _X, 0 ).
```

```
simplify( _X * 0, 0 ).
```

```
simplify( - - X, X ).
```

```
simplify( - 0 , 0 ).
```

```
simplify( -X * Y, - ( X * Y ) ).
```

```
simplify( X * -Y, - ( X * Y ) ).
```

Simplification in Prolog (2)

Try to make the outermost, first possible replacement:

```
outermost( Expr, Res ) :- simplify( Expr, Res ), !.
```

```
outermost( Expr, Res ) :-  
    Expr =.. [ F | ExprList ],  
    outermostlist( ExprList, ResList ),  
    Res =.. [ F | ResList ].
```

```
outermostlist( [], [] ).  
outermostlist( [ E1 | R1 ], [ E2 | R2 ] ) :-  
    outermost( E1, E2 ),  
    outermostlist( R1, R2 ).
```

Simplification in Prolog (3)

Apply outermost as long as possible:

```
normalize( X, Z ) :-  
    outermost( X, Y ), X \= Y,  
    !,  
    normalize( Y, Z ).  
  
normalize( X, X ).
```

Simplification in C++

What is the best signature (most easy to use, and minimizing number of reallocations)?

1. `mathexpr outermost( mathexpr expr )`
2. `mathexpr outermost( const mathexpr& expr )`
3. `std::optional< mathexpr >`
 `outermost( const mathexpr& expr )`
4. `void outermost( mathexpr& expr )`

1,2 are essentially the same. **3** results in ugly code (next slide). **4** is also possible. **1** is best by far.

Problems with (3)

This is the unary case for (3):

```
case sel_neg:
case sel_recipr:
    { std::optional< mathexpr > res;
      auto ar1 = expr. view_arith1( );
      auto e = outermost( repl, ar1. sub( ));
      if( e. has_value( ))
      { if( !res. has_value( ))
        { res = expr;
          res. value( ). view_arith1( ).
                                update_sub( e. value( ));
        }
      }
      return res;
    }
```

(1) is nice

```
case sel_neg:
case sel_recipr:
    { auto ar1 = expr. view_arith1( );
      ar1. update_sub( outermost( repl, ar1. sub( ) ));
    }
    return expr;

case sel_sub:
case sel_truediv:
    { auto ar2 = expr. view_arith2( );
      ar2. update_sub1( outermost( repl, ar2. sub1( ) ));
      ar2. update_sub2( outermost( repl, ar2. sub2( ) ));
    }
    return expr;
```

```
case sel_add:
case sel_mul:
{
    auto arN = expr. view_arithN( );
    for( size_t i = 0; i != arN. size( ); ++ i )
        arN. update_sub( i,
                        outermost( repl, arN. sub(i)) );
}
return expr;
```


Signature of Outermost

```
template< typename R >
concept rewrite_rule =
    requires( R repl, mathexpr expr )
    { { repl( expr ) } -> std::same_as< mathexpr > ; };

template< rewrite_rule R >
mathexpr outermost( R&& repl, mathexpr expr )
{
    {
        mathexpr expr2 = repl( expr );
        if( !expr2. strongly_equal_to( expr ))
            return expr2;
    }
    // Here come the switch and the recursive cases.
}
```

All is Well?

Not Everything is Well!

`outermost` will always reallocate when there is a change in a subtree, even when the tree is not shared.

If you pass a subtree to a function, it is always shared.

In order to avoid this, one needs to get unique ownership of the subterm.

Extract Methods

Pass ownership when taking a subtree of a tree that is not shared.
Otherwise, give out a copy:

```
mathexpr extr_X( expr newval )  
    // Assuming X is below pnt  
  
    if pnt -> nruses == 1;  
        return std::move( ptr -> X );  
    else  
        return ( ptr -> X );
```

This is the perfect solution.

outermost Improved (Perfect?)

This is the signature:

```
template< typename R >
concept rewrite_rule =
    requires( R repl, mathexpr expr )
    { { repl. nruses } -> std::same_as< uint64_t& > ;
      { repl( expr ) } -> std::same_as< mathexpr > ;
    };
```

outermost Improved (2)

```
template< rewrite_rule R >
mathexpr outermost( R&& repl, mathexpr expr )
{
    {
        auto u = repl. nruses;
        expr = repl( std::move( expr ) );
        if( u != repl. nruses )
            return expr;
    }
}
```

Comparing the old tree with the new tree is not possible, because for that, you need to keep the original tree, which means sharing. Because of this, we added a counter to the `rewrite_rule`.

outermost Improved (3)

Code remains nice:

```
case sel_neg:
case sel_recipr:
    { auto ar1 = expr. view_arith1( );
      ar1. update_sub( outermost( repl, ar1. extr_sub( )))
    }
    return expr;

case sel_sub:
case sel_truediv:
    { auto ar2 = expr. view_arith2( );
      ar2. update_sub1( outermost( repl, ar2.extr_sub1()))
      ar2. update_sub2( outermost( repl, ar2.extr_sub2()))
    }
    return expr;
```

outermost Improved (4)

```
case sel_add:
case sel_mul:
{
    auto arN = expr. view_arithN( );
    for( size_t i = 0; i != arN. size( ); ++ i )
        arN. update_sub( i,
                        outermost( repl, arN. extr_sub(i)) );
}
return expr;
```


How Nice it would be to have Refinement Types

We could get rid of the ugly casts.

In Prolog, we write: `simplify( - - X, X )`. In C^{++} , we have:

```
if( expr. sel( ) == sel_neg )
{
    auto ar1 = expr. view_arith1( );
    if( ar1. sub( ). sel( ) == sel_neg )
    {
        auto ar1a = ar1. sub( ). view_arith1( );
        return ar1a. sub( );
    }
}
```

Long, and incomprehensible.

Refinement Types (2)

Purpose of the `views` is to please the static type system. `ar1` is just `expr`, but with a changed type.

We know that the cast is redundant but the type system of C^{++} does not. Without it, one could write:

```
if( expr. sel( ) == sel_neg &&
    expr. sub( ). sel( ) == sel_neg )
{
    return expr. sub( ). sub( );
}
```

I think that would be acceptable.

Refinement Types (3)

Similar code occurs all the time, when you have some kind of sum type:

```
std::optional< std::string > str;  
if( ... ) str = "CppNow!";  
  
if( str. has_value( ))  
    std::cout << str. value( );  
    // .value( ) is redundant! Should be just str.
```

Refinement Types (4)

An example with `std::variant` :

```
std::variant< std::string, int > smg =  
    "in Muenchen steht ein Hofbraeuhaus";  
  
if( ... ) smg = 32168;  
  
if( holds_alternative< std::string > ( smg ))  
    std::string s = get< std::string > ( smg );  
  
if( holds_alternative< int > ( smg ))  
    int nr = get< int > ( smg );
```

It would be possible to write just `smg` in both cases, without `get`.

Lists

Could one add lists (in addition to vector)?

1. Is it useful?
2. Is it possible?

As for **(2)**, the problem is value semantics. Lists exist, because insertion/deletion from the middle is $\Theta(1)$. This can be obtained with iterators only. Iterators break value semantics.

Updating/extracting is not possible with iterators.

Possible solution: Create an iterator type that contains a pointer to the list.

Is that possible?

Thanks!

Thanks to Aleksandra Kireeva, Kaster Kumarbek, Akhmetzhan Kussainov, Dina Muktubayeva, Olzhas Zhangel'dinov, and Olzhas Zhumabek.

I also thank Nazarbayev University for supporting this project through the Faculty Development Competitive Research Grant Program (FDCRGP), grant number 021220FD1651.